

深度學習作業四

1. Experiment with different window sizes and steps. Train the model using 3 different combinations of window size and step. Evaluate the Mean Squared Error (MSE) for each configuration. Report the MSEs using a table and analyze the results.

	window	step	MSE
0	30	1	0.134416
1	60	1	0.047660
2	60	5	0.019918

將觀察視窗由 30 擴大至 60 天，MSE 由 0.134 顯著降至 0.048，顯示 LSTM 需要較長歷史序列才能完整捕捉價格趨勢。進一步在 60 天視窗下把步長由 1 增至 5，可再將 MSE 降到 0.020；較大的步長減少了高度重複的鄰近樣本，降低過擬合，同時保留主要波動訊號，整體而言「視窗 60、步長 5」提供最佳誤差表現。

2. Include 'Volume' as an additional input feature in your model. Discuss the impact of incorporating 'Volume' on the model's performance.

	combo	MSE
2	OHLC	0.011675
0	Close	0.035648
1	Close+Volume	0.094926
3	OHLC+Volume	0.126899

在僅用 Close 時，模型只需擬合單一序列，MSE 為 0.0356。當把 Volume 加進去 (Close + Volume)，誤差上升至 0.0949；同樣地，在 OHLC 基礎上再加 Volume 也讓 MSE 由 0.0117 惡化到 0.1269。這顯示 Volume 對結果的解釋能力有限，加入容易使網路注意力分散到雜訊，導致過擬合與泛化下降。

Explore and report on the best combination of input features that yields the best MSE. Describe the reasons for your attempts and analyze the final, optimal input combination. 經過四次嘗試 OHLC (Open、High、Low、Close) 表現最佳，OHLC 兼具 High/Low 讓模型捕捉日內波動範圍、Open 價反映隔夜消息對開盤價的衝擊，維護資訊量與穩定性，是目前本資料集下的最優輸入組合。

3. Analyze the performance of the model with and without normalized inputs in Lab 4. You can use experimental results or external references (which must be cited) to support your conclusions on whether normalization improves the model's performance.

Without normalization MSE = 3499.0507
With Z-score MSE = 0.1381

經由實驗結果得知，未經任何縮放的 OHLC 價格直接輸入 LSTM 時，驗證 MSE=3499.05，而採用 Z-score 標準化後僅 0.1381。原因在於原始價格跨度大，梯度計算易爆炸；標準化把每一欄轉為零均值、單位變異，使各特徵對參數更新的影響均衡、收斂曲線平滑，網路得以使用較大學習率快速逼近最小化目標。

4. Why should the window size be less than the step size in Lab 4?

Do you think this is correct?

Window size = 10 < step = 15，這樣的作法可以產生不重疊切片、降低自相關影響，雖然會犧牲樣本數，但可換取樣本獨立性。

這樣做並沒有錯誤，然而這種做法並不常見，因為在時間序列建模中，window size $\geq$ step size，這樣相鄰樣本會重疊一部分，既增加訓練資料量，也保留時間連續性，因此如果需要最大化訓練資料量、學習更細微的時序模式，則需進行修改。

5. Describe one method for data augmentation specifically applicable to time-series data.

抖動（Jittering）是常見的時間序列資料擴增方法，其透過向原始序列的每個時間步加入隨機噪聲以生成新樣本。具體而言，對於長度 T 的序列 $x = \{x_1, \dots, x_T\}$ ，在每個位置加入 $\varepsilon_t \sim N(0, \sigma^2)$ 得到 $x'_t = x_t + \varepsilon_t$ 。

此方法能模擬真實資料中的測量誤差與外部干擾，使模型在雜訊環境下仍保持穩健表現，降低過擬合風險並提升泛化能力。文獻指出，抖動是最簡單且廣泛使用的時序擴增技術之一，對感測器及金融等領域資料尤為有效。

參考資料: Data Augmentation techniques in time series domain: a survey and taxonomy

6. Discuss how to handle window size during inference in different model architectures.

(a) Convolution-based models

卷積網路的「視窗」等同於感受野 (receptive field)，即核大小與層數決定可見序列長度。推論時，可直接將整段輸入送入全卷積結構，不需額外切片；若記憶體有限，則採滑動窗口 (window) 分批卷積，每次移動步長等於 stride，最後拼接或平均多個窗口的輸出。

(b) Recurrent-based models

RNN/LSTM 本身能流式接收任意長度序列，故推論理論上不用固定 window。

但為了控制計算開銷與隱藏狀態衰減，常實作「截斷」：只保留最近 N 步作為窗口，並將上次隱藏狀態加以初始化。對超長序列，還可分段推理；每段輸出最後的 hidden state 作為下一段的輸入初始值。

(c) Transformer-based models

Transformer 定義了最大序列長度 L ，推論時必須將輸入裁切或填充至 L 。若原始序列超過 L ，可採「重疊滑窗」分塊推理，每塊包含前一塊的尾部上下文；或利用稀疏/分層注意力與鍵值緩存 (key-value cache) 機制，將過往塊的注意力資訊保留，以支援長序列或流式推論。